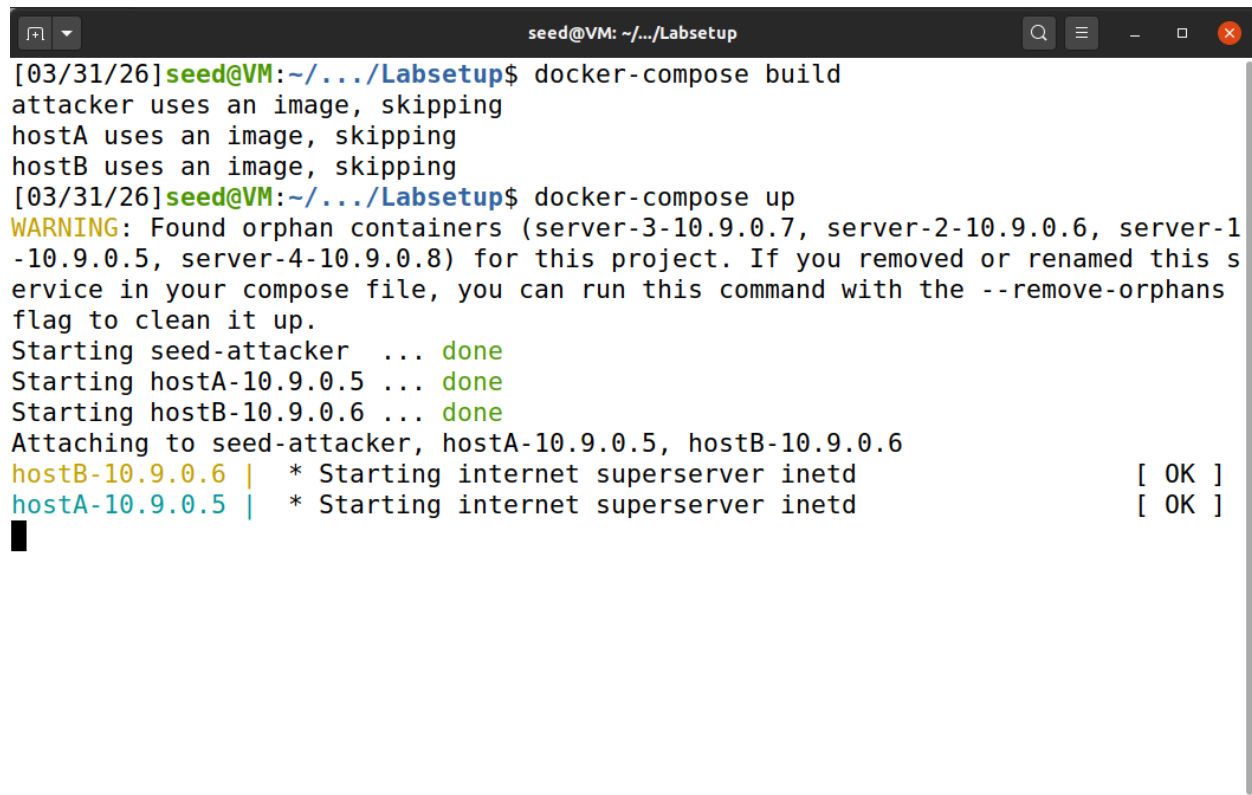


Duy Hoang
CSIT460
Lab 3

Part 1 - Packet Sniffing and Spoofing Lab

1.)The lab began with executing docker-compose build followed by docker-compose up within the Labsetup folder. Out came the containers, now live, forming the needed network layout for the task ahead. Inside this setup, per the provided PDF, sit three devices sharing one local area network - attacker rests at 10.9.0.1, Host A lives at 10.9.0.5, while Host B holds position at 10.9.0.6.

A terminal window titled 'seed@VM: ~/.../Labsetup' showing the execution of docker-compose commands. The window has a dark background with a search icon, a menu icon, and window control buttons in the title bar. The terminal output shows the build process for 'attacker', 'hostA', and 'hostB' being skipped as images are already present. Then, 'docker-compose up' is executed, showing a warning about orphan containers and successful startup for 'seed-attacker', 'hostA-10.9.0.5', and 'hostB-10.9.0.6'. Finally, it shows 'Attaching to seed-attacker, hostA-10.9.0.5, hostB-10.9.0.6' and the start of 'internet superserver inetd' on both hostB and hostA, each with a '[OK]' status.

```
[03/31/26]seed@VM:~/.../Labsetup$ docker-compose build
attacker uses an image, skipping
hostA uses an image, skipping
hostB uses an image, skipping
[03/31/26]seed@VM:~/.../Labsetup$ docker-compose up
WARNING: Found orphan containers (server-3-10.9.0.7, server-2-10.9.0.6, server-1-10.9.0.5, server-4-10.9.0.8) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
Starting seed-attacker ... done
Starting hostA-10.9.0.5 ... done
Starting hostB-10.9.0.6 ... done
Attaching to seed-attacker, hostA-10.9.0.5, hostB-10.9.0.6
hostB-10.9.0.6 | * Starting internet superserver inetd      [ OK ]
hostA-10.9.0.5 | * Starting internet superserver inetd      [ OK ]
```

2.)With the containers up and going, dockps helped spot their IDs along with labels. That move confirmed the attacker, Host A, and Host B were live. Names pulled here turned useful - needed them to launch shell access inside each container down the line.

```
seed@VM: ~/.../Labsetup$ dockps
8d37d858d084  hostA-10.9.0.5
db6ec62e63c6  hostB-10.9.0.6
19d3f8a24067  seed-attacker
[03/31/26]seed@VM:~/.../Labsetup$
```

3.)From there, I started docksh seed-attacker to access the attacker container. That box handled all packet capture and manipulation work.

```
seed@VM: ~/.../Labsetup$ docksh seed-attacker
root@VM:/#
```

4.)From inside Host A's container, I set up a way to send data later during the exercise. That matters since the attack container must see conversations between devices to catch live traffic. If Host A and Host B stay quiet, the sniffer picks up empty air instead of real movement.

```
seed@VM: ~/.../Labsetup$ docksh hostA-10.9.0.5
root@8d37d858d084:/#
```

5.) After checking, the proper network interface inside the VM turned out to be br-a6ae5d5e5628 - linked directly to the 10.9.0.1 network. Listening happens here since the sniffer must attach to this specific point. Wrong choice means silence or random data flowing past without meaning.

```
seed@VM: ~/.../Labsetup
19d3f8a24067 seed-attacker
[03/31/26]seed@VM:~/.../Labsetup$ ifconfig
br-a6ae5d5e5628: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.9.0.1 netmask 255.255.255.0 broadcast 10.9.0.255
    inet6 fe80::42:85ff:fe13:83f7 prefixlen 64 scopeid 0x20<link>
    ether 02:42:85:13:83:f7 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Lab Task 1

6.) From the beginning of Lab Task 1, work began in the shared volumes directory. That spot matters - changes made here show up both on the VM and in the attacker container. As shown in the PDF, the VM's /volumes maps directly to /volumes in the container. Files placed there appear instantly across both systems. Working this way simplifies updating Python scripts before execution. Because of this setup, moving between editing and testing feels smooth.

```
seed@VM: ~/.../volumes
GNU nano 4.8 mycode.py
#!/usr/bin/env python3
from scapy.all import *

a= IP()
a.show()
~
~
~
```

7.) A file named mycode.py appeared inside the shared folder - just a quick way to see if Scapy ran properly. From there, a short Python script took shape: it pulled in Scapy, built an IP object, then listed out the fields. That little run acted like a signal, quiet and clear, showing everything stood ready before stepping into packet capture and imitation tasks.

```
seed@VM: ~/.../Labsetup
boot etc lib lib64 media opt root sbin sys usr volumes
root@VM:/# cd volumes
root@VM:/volumes# ls
mycode.py
root@VM:/volumes# python3 mycode.py
File "mycode.py", line 6
~
^
SyntaxError: invalid syntax
root@VM:/volumes# python3 mycode.py
####[ IP ]####
version    = 4
ihl        = None
tos        = 0x0
len        = None
id         = 1
flags      =
frag       = 0
ttl        = 64
proto      = hopopt
chksum     = None
src        = 127.0.0.1
dst        = 127.0.0.1
\options   \
```

8.) Running python3 mycode.py showed the IP header details on screen. Only after seeing those values did I know Scapy worked right. Inside the attacker container, building and checking packets became possible. The readiness of the Python setup only becomes clear through such a

check.

```
root@VM:/volumes# chmod a+x mycode.py
root@VM:/volumes# ls -la
total 16
drwxrwxr-x 2 seed seed 4096 Mar 31 09:09 .
drwxr-xr-x 1 root root 4096 Mar 30 15:14 ..
-rw-rw-r-- 1 seed seed   0 Dec  5 2020 .gitignore
-rw-rw-r-- 1 seed seed 1024 Mar 31 09:09 .mycode.py.swp
-rwxrwxr-x 1 seed seed   65 Mar 31 09:09 mycode.py
root@VM:/volumes#
```

9.) Later on, the file got execution rights through `chmod a+x mycode.py`. Task files followed for the remaining Part 1 bits. Preparation took up most of this phase, yet it cleared the path - each task lived in its separate Python script.

```
[03/31/26] seed@VM:~/.../volumes$ ls
mycode.py
[03/31/26] seed@VM:~/.../volumes$ touch task1.{1..3}.py
[03/31/26] seed@VM:~/.../volumes$ ls
mycode.py task1.1.py task1.2.py task1.3.py
[03/31/26] seed@VM:~/.../volumes$
```

Task 1.1: Sniffing Packets

10.) I started Task 1.1 by editing `task1.1.py`, slipping in Scapy's packet-sniffing lines. This script had one job: catch traffic through the right bridge port, then spill details using `pkt.show()`. Instead of copying word for word, I matched the layout from the PDF, which outlined `sniff()` with interface settings, filters, plus a function that runs each time a packet arrives.

```
seed@VM: ~/.../volumes
GNU nano 4.8 task1.1.py
#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
    pkt.show()

pkt = sniff(iface='br-a6ae5d5e5628', filter='icmp', prn=print_pkt)
```

Task 1.1A:

11.) Inside the attacker container, I turned task1.1.py into something that could run directly. The moment execution started, it stayed active, simply waiting for incoming packets. That kind of behavior? Normal for a packet sniffer.

```
root@VM:/volumes# chmod a+x task1.1.py
root@VM:/volumes# ll
total 8
-rwxrwxr-x 1 seed seed 65 Mar 31 09:09 mycode.py
-rwxrwxr-x 1 seed seed 151 Mar 31 09:15 task1.1.py
-rw-rw-r-- 1 seed seed 0 Mar 31 09:12 task1.2.py
-rw-rw-r-- 1 seed seed 0 Mar 31 09:12 task1.3.py
root@VM:/volumes#
```

12.) A different terminal window came open just to get some data moving for the sniffer. Inside it, a connection started toward Host A before sending pings over to Host B. Packets now moved back and forth - echo requests followed by replies - all flowing across the test setup. The attacker's container watched one specific port closely, catching every bit that passed through. Even without being involved in the exchange, the traffic still appeared within its view.

```
root@VM:/volumes# chmod a+x task1.1.py
root@VM:/volumes# ll
total 8
-rwxrwxr-x 1 seed seed 65 Mar 31 09:09 mycode.py
-rwxrwxr-x 1 seed seed 151 Mar 31 09:15 task1.1.py
-rw-rw-r-- 1 seed seed 0 Mar 31 09:12 task1.2.py
-rw-rw-r-- 1 seed seed 0 Mar 31 09:12 task1.3.py
root@VM:/volumes# ./task1.1.py
```

13.) Back at the attacker machine, the sniffing tool had already listed out the intercepted packet data. Looking closer, each header appeared just as expected - proof that the ping's ICMP messages got picked up without issue. Clearly, the monitoring software did its job exactly how it should. With the right port access, anyone on the local segment might quietly watch exchanges happening across connected devices.

```

root@8d37d858d084:/# ping 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data.
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.159 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.090 ms
64 bytes from 10.9.0.6: icmp_seq=3 ttl=64 time=0.093 ms
64 bytes from 10.9.0.6: icmp_seq=4 ttl=64 time=0.089 ms
^Z
[2]+  Stopped                  ping 10.9.0.6
root@8d37d858d084:/# █

```

14.) What this part of the task really shows is how packet sniffing works in practice. The attacker does not have to directly participate in the communication to observe it. The sniffer simply listens on the network and records the traffic that passes by. This is one reason packet sniffing is considered a security threat, especially on networks where sensitive data is not encrypted.

```

###[ Ethernet ]###
  dst      = 02:42:0a:09:00:05
  src      = 02:42:0a:09:00:06
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 84
  id       = 15210
  flags    =
  frag     = 0
  ttl      = 64
  proto    = icmp
  chksum   = 0x2b23
  src      = 10.9.0.6
  dst      = 10.9.0.5
  \options \
###[ ICMP ]###
  type     = echo-reply
  code     = 0
  chksum   = 0x4ea6
  id       = 0x2
  seq      = 0x4
###[ Raw ]###
  load     = '\xe8\x91\xcb\x00\x00\x00\x07\x85\x07\x00\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&'()*+,-./01234567'

```

Task 1.1B

15.) Starting with Task 1.1B, the sniffer code got adjusted for tighter filtering. Instead of broad capture, it now zeroes in using a TCP rule targeting one source IP plus port 23 as endpoint. The image shows how the condition changed - now only Telnet data gets recorded. Since live

```
[03/31/26]seed@VM:~/.../volumes$ docksh hostB-10.9.0.6
root@db6ec62e63c6:/# ping 128.230.0.11
PING 128.230.0.11 (128.230.0.11) 56(84) bytes of data.
^X
```

```
^[
^Z
[1]+  Stopped                  ping 128.230.0.11
root@db6ec62e63c6:/#
```

```

###[ Ethernet ]###
dst      = 02:42:85:13:83:f7
src      = 02:42:0a:09:00:06
type     = IPv4
###[ IP ]###
version  = 4
ihl      = 5
tos      = 0x0
len      = 84
id       = 50347
flags    = DF
frag     = 0
ttl      = 64
proto    = icmp
chksum   = 0xaeaf
src      = 10.9.0.6
dst      = 128.230.0.11
\options \
###[ ICMP ]###
type     = echo-request
code     = 0
chksum   = 0xa96d
id       = 0x3
seq      = 0x22
###[ Raw ]###
load     = '\x1b\x94\xcbi\x00\x00\x00\x00\x9f\x9c\t\x00\x00\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\n!#$%&'()*+,-./01234567'

```

17.) Starting with Task 1.2, the focus shifted to the spoofing script named task1.2.py. Inside that file, an IP object came first, its target address set right after. Then, an ICMP structure took shape separately. These two pieces joined together, forming a single packet ready to go. Transmission happened through send(). The fake origin got assigned 10.0.2.3 - this number stood out clearly once results appeared.

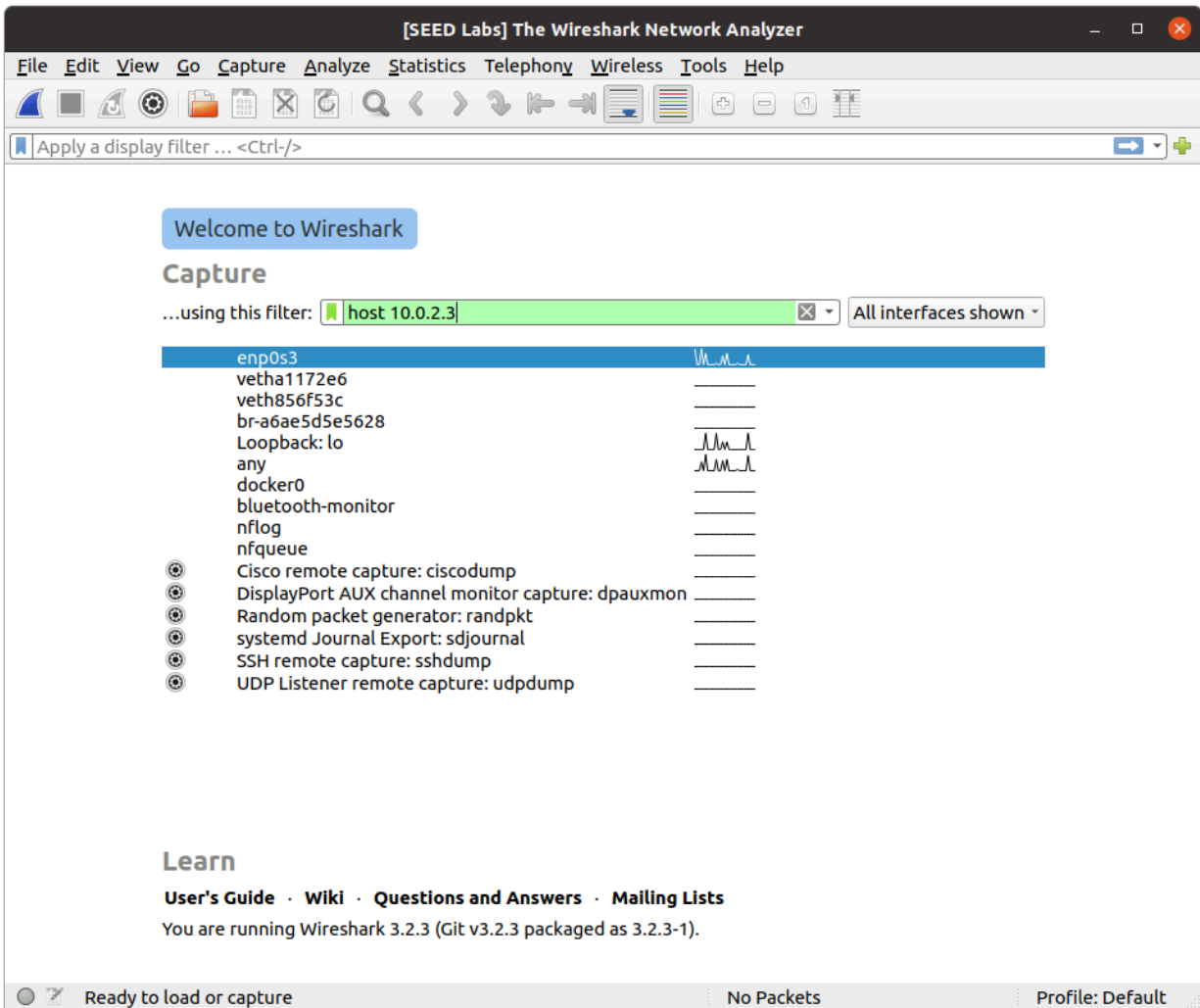
```
seed@VM: ~/.../volumes
GNU nano 4.8 task1.2.py
#!/usr/bin/env python3

from scapy.all import *
a = IP()
a.dst = '10.0.2.3'
b = ICMP()
p = a/b

ls (a)

send(p)
■
```

18.)With the spoofing script waiting, I started Wireshark first, picking the right network port to watch data move. Seeing it happen on screen makes confirmation simpler than guessing. A live capture shows each packet leaving with a forged IP, not the actual one tied to my machine.



19.) Out of nowhere, task1.2.py became executable. From inside the attacker container, execution followed. A single packet flew out. Right then, the terminal showed proof - transmission

complete.

```
^Croot@VM:/volumes# chmod a+x task1.2.py
```

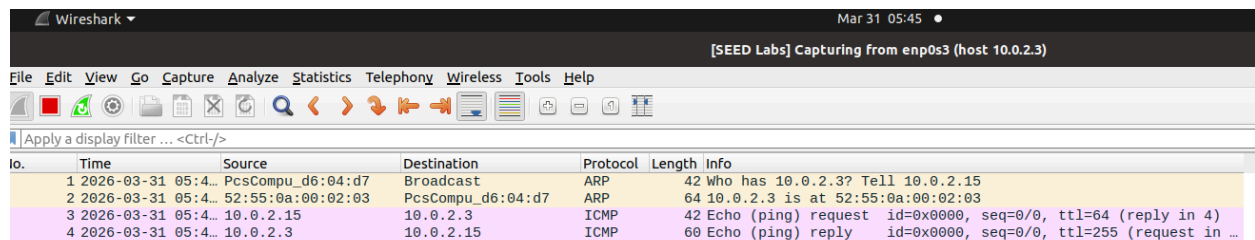
```
root@VM:/volumes# ./task1.2.py
```

```
version      : BitField  (4 bits)          = 4              (4)
ihl          : BitField  (4 bits)          = None           (None)
tos          : XByteField              = 0              (0)
len          : ShortField              = None           (None)
id           : ShortField              = 1              (1)
flags        : FlagsField  (3 bits)       = <Flag 0 (>)    (<Flag 0 (>))
frag         : BitField  (13 bits)        = 0              (0)
ttl          : ByteField                = 64             (64)
proto        : ByteEnumField            = 0              (0)
chksum       : XShortField              = None           (None)
src          : SourceIPField            = '10.0.2.15'    (None)
dst          : DestIPField              = '10.0.2.3'     (None)
options      : PacketListField          = []             ([])
```

```
Sent 1 packets.
```

```
root@VM:/volumes# █
```

20.) Looking at Wireshark, the fake packet showed up right away. That meant the altered ICMP message actually made it onto the network. Proof sits in front of me - this trick works beyond textbooks. Changing values in the IP header let the data look like it started somewhere else entirely.



The image shows a Wireshark packet capture window. The title bar indicates it is capturing from 'enp0s3 (host 10.0.2.3)'. The packet list shows four packets:

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-03-31 05:4...	PcsCompu_d6:04:d7	Broadcast	ARP	42	Who has 10.0.2.3? Tell 10.0.2.15
2	2026-03-31 05:4...	52:55:0a:00:02:03	PcsCompu_d6:04:d7	ARP	64	10.0.2.3 is at 52:55:0a:00:02:03
3	2026-03-31 05:4...	10.0.2.15	10.0.2.3	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 4)
4	2026-03-31 05:4...	10.0.2.3	10.0.2.15	ICMP	60	Echo (ping) reply id=0x0000, seq=0/0, ttl=255 (request in ...)

Task 1.3: Traceroute

21.) For Task 1.3, I moved to the traceroute program in task1.3.py. I wrote the code so it would send packets toward the destination 8.8.4.4 while changing the TTL value. The point of this task is to use Scapy to imitate how traceroute works. The PDF explains that by starting with a low TTL and increasing it one step at a time, I can observe which router responds at each stage and estimate the path taken by the packets.

```
seed@VM: ~/.../volumes
GNU nano 4.8 task1.3.py
#!/usr/bin/env python3

from scapy.all import *
import sys

a = IP()
a.dst = '8.8.4.4'
a.ttl = int(sys.argv[1])
b = ICMP()
#send(a/b)
a = sr1(a/b)
```

22.) Once the traceroute file could run, multiple attempts followed using varied TTL settings. From each try, a fresh source IP came back - every one marking yet another step forward. The endpoint began showing up as the sender only after those steps unfolded. That shift meant the message arrived exactly where it needed to go.

```
root@VM:/volumes# chmod a+x task1.3.py
root@VM:/volumes# ./task1.3.py
Traceback (most recent call last):
  File "./task1.3.py", line 8, in <module>
    a.ttl = int(sys.argv[1])
IndexError: list index out of range
root@VM:/volumes# ./task1.3.py 3
Begin emission:
Finished sending 1 packets.
.*
Received 2 packets, got 1 answers, remaining 0 packets
root@VM:/volumes#
```

23.) Looking at Wireshark during the traceroute job, the same sequence showed up again. The last packet stood out - source and target swapped places since it came straight from the endpoint now. What I saw on screen lined up with what the terminal had printed. That match made clear: the traceroute method ran correctly.

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-03-31 05:5...	10.0.2.15	8.8.4.4	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=3 (reply in 2)
2	2026-03-31 05:5...	8.8.4.4	10.0.2.15	ICMP	60	Echo (ping) reply id=0x0000, seq=0/0, ttl=255 (request in ...)

24.) When the lab finished, the environment was turned off using `docker-compose down`. Containers halted without issues, marking the task complete. Part 1 made clear how watching traffic differs from making fake packets myself. Listening carefully on a chosen interface revealed what attackers see during sniffing exercises. Changing small details in packet headers suddenly let me pretend to be someone else entirely. Last of all, during the traceroute part, by using Scapy, I saw how changing TTL exposes each stop data across a network.

```

seed@VM: ~/.../Labsetup
[03/31/26]seed@VM:~/.../Labsetup$ ls
docker-compose.yml  volumes
[03/31/26]seed@VM:~/.../Labsetup$ docker-compose down
Stopping hostA-10.9.0.5 ... done
Stopping hostB-10.9.0.6 ... done
Stopping seed-attacker ... done
WARNING: Found orphan containers (server-1-10.9.0.5, server-3-10.9.0.7, server-4
-10.9.0.8, server-2-10.9.0.6) for this project. If you removed or renamed this s
ervice in your compose file, you can run this command with the --remove-orphans
flag to clean it up.
Removing hostA-10.9.0.5 ... done
Removing hostB-10.9.0.6 ... done
Removing seed-attacker ... done
Removing network net-10.9.0.0
[03/31/26]seed@VM:~/.../Labsetup$

seed@VM: ~/.../Labsetup
[03/31/26]seed@VM:~/.../Labsetup$ docker-compose build
attacker uses an image, skipping
hostA uses an image, skipping
hostB uses an image, skipping
[03/31/26]seed@VM:~/.../Labsetup$ docker-compose up
WARNING: Found orphan containers (server-3-10.9.0.7, server-2-10.9.0.6, server-1
-10.9.0.5, server-4-10.9.0.8) for this project. If you removed or renamed this s
ervice in your compose file, you can run this command with the --remove-orphans
flag to clean it up.
Starting seed-attacker ... done
Starting hostA-10.9.0.5 ... done
Starting hostB-10.9.0.6 ... done
Attaching to seed-attacker, hostA-10.9.0.5, hostB-10.9.0.6
hostB-10.9.0.6 | * Starting internet superserver inetd
hostA-10.9.0.5 | * Starting internet superserver inetd
seed-attacker | root@VM:/# seed-attacker exited with code 137
hostA-10.9.0.5 exited with code 137
hostB-10.9.0.6 exited with code 137
[03/31/26]seed@VM:~/.../Labsetup$

xError: list index out of range
@VM:/volumes# ./task1.3.py 3
n emission:
shed sending 1 packets.
ived 2 packets, got 1 answers, remaini

```

Part 2 Firewall Exploration Lab

1.) Got things ready by building the lab first. Inside Labsetup, launching dcbuild made Docker assemble every required container image. Without that, there would be no working router or hosts at all. Testing firewalls only moves forward once those pieces run properly.

```
seed@VM: ~/.../Labsetup$ dcbuild
[03/31/26]seed@VM:~/.../Labsetup$ dcbuild
HostA uses an image, skipping
Host1 uses an image, skipping
Host2 uses an image, skipping
Host3 uses an image, skipping
Building Router
Step 1/2 : FROM hands-on-security/seed-ubuntu:large
--> cecb04fbf1dd
Step 2/2 : RUN apt-get update      && apt-get install -y kmod      && apt-get clean
--> Running in ebb098c3b581
Get:1 http://archive.ubuntu.com/ubuntu focal InRelease [265 kB]
Get:2 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Get:3 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64 Packages [33.1 kB]
Get:4 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:5 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 Packages [4801 kB]
Get:6 http://archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Get:7 http://archive.ubuntu.com/ubuntu focal/universe amd64 Packages [11.3 MB]
Get:8 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [4432 kB]
Get:9 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages [1308 kB]
Get:10 http://archive.ubuntu.com/ubuntu focal/restricted amd64 Packages [33.4 kB]
Get:11 http://archive.ubuntu.com/ubuntu focal/main amd64 Packages [1275 kB]
Get:12 http://archive.ubuntu.com/ubuntu focal/multiverse amd64 Packages [177 kB]
Get:13 http://archive.ubuntu.com/ubuntu focal-updates/universe amd64 Packages [1599 kB]
Get:14 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages [4920 kB]
Get:15 http://archive.ubuntu.com/ubuntu focal-updates/restricted amd64 Packages [4998 kB]
Get:16 http://archive.ubuntu.com/ubuntu focal-updates/multiverse amd64 Packages [36.8 kB]
Get:17 http://archive.ubuntu.com/ubuntu focal-backports/universe amd64 Packages [28.6 kB]
Get:18 http://archive.ubuntu.com/ubuntu focal-backports/main amd64 Packages [55.2 kB]
Fetched 35.7 MB in 5s (6554 kB/s)
```

2.) When the build ended, dcup launched every container without delay. Right after that process finished, everything ran quietly behind the scenes - access opened up to the outer machine, the inner machines, plus the routing device linking both networks. The layout fits exactly what appears in the firewall lab document: a bridge forms between the external 10.9.0.0/24 space and the internal 192.168.60.0/24 segment through that central router.

```
[03/31/26]seed@VM:~/.../Labsetup$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating network "net-192.168.60.0" with the default driver
WARNING: Found orphan containers (server-2-10.9.0.6, server-1-10.9.0.5, server-4-10.9.0.8, server-3-10.9.0.7) for this project. If you remove
d or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
Creating seed-router      ... done
Creating hostA-10.9.0.5    ... done
Creating host2-192.168.60.6 ... done
Creating host3-192.168.60.7 ... done
Creating host1-192.168.60.5 ... done
Attaching to host1-192.168.60.5, host2-192.168.60.6, host3-192.168.60.7, hostA-10.9.0.5, seed-router
host1-192.168.60.5 | * Starting internet superserver inetd      [ OK ]
host2-192.168.60.6 | * Starting internet superserver inetd      [ OK ]
host3-192.168.60.7 | * Starting internet superserver inetd      [ OK ]
hostA-10.9.0.5 | * Starting internet superserver inetd      [ OK ]
seed-router | * Starting internet superserver inetd      [ OK ]
```

3.) Just to check everything worked, I fired up a second terminal window then typed dockps. Out popped the list - container names alongside their IDs, clear as day. That detail mattered since

what came next demanded jumping straight into the router container, home of every firewall rule floating around.

```
seed@VM: ~/.../Labsetup
[03/31/26] seed@VM: ~/.../Labsetup$ dockps
6223256825f1  host3-192.168.60.7
3cb4824b7661  host1-192.168.60.5
ad29d866c553  host2-192.168.60.6
2cb892bcb1c   hostA-10.9.0.5
5a5d382d0a5c  seed-router
[03/31/26] seed@VM: ~/.../Labsetup$
```

4.) From the name that appeared in dockps, access to the router container began via docksh seed-router. What matters here, especially in Part 2, is how protection isn't handled directly on each host. The filtering happens elsewhere - on the router, which manages traffic flow between outside and inside networks.

```
seed@VM: ~/.../Labsetup
[03/31/26] seed@VM: ~/.../Labsetup$ docksh seed-router
root@5a5d382d0a5c:/# ls
bin  dev  home  lib32  libx32  mnt  proc  run  srv  tmp  var
boot  etc  lib  lib64  media  opt  root  sbin  sys  usr  volumes
root@5a5d382d0a5c:/#
```

Task 2: Experimenting with Stateless Firewall Rules

Task2.A: Protecting the Router

5.) One way to start: I used iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT for the initial rule. Since then, the router can receive ICMP echo-request messages - these come from pings sent by other devices. Ping attempts fail later if this isn't set, especially after turning on the block-everything default. So now it listens for those signals instead of ignoring them outright. That command makes sure a basic check like ping still works.

```

root@5a5d382d0a5c:/# iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
root@5a5d382d0a5c:/# -t filter -L -n
bash: -t: command not found
root@5a5d382d0a5c:/# iptables -t filter -L -n
Chain INPUT (policy ACCEPT)
target      prot opt source                destination
ACCEPT      icmp -- 0.0.0.0/0              0.0.0.0/0          icmp type 8

Chain FORWARD (policy ACCEPT)
target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target      prot opt source                destination
root@5a5d382d0a5c:/# █

```

6.)After that came iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT. This line tells the system it can transmit responses when someone pings it. When a signal arrives asking for feedback, approval exists to respond. Missing this entry means hearing the knock - yet staying silent.

```

root@5a5d382d0a5c:/# iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT
root@5a5d382d0a5c:/# iptables -t filter -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source                               destination
ACCEPT     icmp --  0.0.0.0/0                             0.0.0.0/0          icmptype 8

Chain FORWARD (policy ACCEPT)
target     prot opt source                               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                               destination
ACCEPT     icmp --  0.0.0.0/0                             0.0.0.0/0          icmptype 0
root@5a5d382d0a5c:/# █

```

7.)Once ping was permitted, the next move shifted control - default output became deny via iptables -P OUTPUT DROP. Outgoing packets now hit a wall unless a clear pass exists. Restriction tightened without warning. The router grew strict, silent, firm.

```

root@5a5d382d0a5c:/# iptables -P OUTPUT DROP
root@5a5d382d0a5c:/# iptables -P INPUT DROP
root@5a5d382d0a5c:/# iptables -t filter -L -n
Chain INPUT (policy DROP)
target      prot opt source                destination
ACCEPT      icmp -- 0.0.0.0/0              0.0.0.0/0          icmptype 8

Chain FORWARD (policy ACCEPT)
target      prot opt source                destination

Chain OUTPUT (policy DROP)
target      prot opt source                destination
ACCEPT      icmp -- 0.0.0.0/0              0.0.0.0/0          icmptype 0
root@5a5d382d0a5c:/# █

```

8.) After that, I used iptables -P INPUT DROP to make the system discard all incoming traffic by default. Now, unless a connection is specifically approved, it won't go through. From here on, only what I say can enter - everything else gets shut out. That kind of setup refuses most attempts right away, letting just the needed paths stay open. Blocking first, allowing after - this way keeps things tight without extra noise.

```
root@5a5d382d0a5c:/# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
12: eth0@if13: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:0a:09:00:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.11/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
16: eth1@if17: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:c0:a8:3c:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.60.11/24 brd 192.168.60.255 scope global eth1
        valid_lft forever preferred_lft forever
root@5a5d382d0a5c:/#
```

9.) A quick look at the output from iptables -L -n told me if things went right. Shown in the image, each chain now holds the new entries - both ICMP lines appear exactly where they should. The defaults shifting to DROP wasn't accidental either; that change stuck too. Seeing them there meant these weren't just keystrokes vanishing into memory. Rules active means real protection started working the moment they loaded. Proof sits in the terminal scroll - live, visible, doing their job.

```
seed@VM: ~/.../Labsetup
[03/31/26] seed@VM: ~/.../Labsetup$ docksh host3-192.168.60.7
root@6223256825f1:/# ping 192.168.60.11
PING 192.168.60.11 (192.168.60.11) 56(84) bytes of data.
64 bytes from 192.168.60.11: icmp_seq=1 ttl=64 time=2.94 ms
64 bytes from 192.168.60.11: icmp_seq=2 ttl=64 time=0.051 ms
64 bytes from 192.168.60.11: icmp_seq=3 ttl=64 time=0.110 ms
64 bytes from 192.168.60.11: icmp_seq=4 ttl=64 time=0.184 ms
^C
--- 192.168.60.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3039ms
rtt min/avg/max/mdev = 0.051/0.822/2.944/1.225 ms
root@6223256825f1:/#
```

10.) Inside the router container, I used ip addr to check settings, spotting two distinct IPs assigned - one per network. One sits at 10.9.0.11 facing outward, while 192.168.60.11 handles the internal

side. That setup proves the router bridges both networks directly. Because of this role, any firewall rule applied impacts traffic on either end. Position defines function - here, control spans across.

```
seed@VM: ~/.../Labsetup
[03/31/26]seed@VM:~/.../Labsetup$ docksh host1-192.168.60.5
root@3cb4824b7661:/# telnet 19.9.0.11
Trying 19.9.0.11...
^C
root@3cb4824b7661:/#
```

11.) From behind the screen, testing began once settings were applied. Reaching out to the router happened through a ping attempt followed by an effort using telnet. What turned up in images revealed something clear - ping signals got through since they matched what the setup allowed. Telnet simply stopped cold due to missing TCP permissions. Only one kind of path stayed open on purpose. Protection took shape not by opening doors but by shutting them tight. Limited entry became the shield around the device, exactly as needed for Task 2.A's core aim.

```
root@5a5d382d0a5c:/# iptables -F
root@5a5d382d0a5c:/# iptables -P OUTPUT ACCEPT
root@5a5d382d0a5c:/# iptables -P INPUT
iptables v1.8.4 (legacy): -P requires a chain and a policy
Try `iptables -h' or 'iptables --help' for more information.
root@5a5d382d0a5c:/# iptables_P INPUT
bash: iptables_P: command not found
root@5a5d382d0a5c:/# iptables -P INPUT ACCEPT
root@5a5d382d0a5c:/#
```

Task 2.B: Protecting the Internal Network

12.) First up, the prior setup got wiped clean with iptables -F before moving forward. Old protections had to go - default policies were set back just in case. That cleanup matters more than it looks. Now the goal shifts entirely: not guarding the router anymore. Protection focuses inward from here on out. The inside systems are what need shielding now.

```
root@5a5d382d0a5c:/# iptables -A FORWARD -i eth0 -p icmp --icmp-type echo-request -j DROP
```

```
root@5a5d382d0a5c:/# iptables -L -n
```

```
Chain INPUT (policy ACCEPT)
```

```
target      prot opt source                      destination
```

```
Chain FORWARD (policy ACCEPT)
```

```
target      prot opt source                      destination      icmptype 8
DROP        icmp -- 0.0.0.0/0                    0.0.0.0/0
```

```
Chain OUTPUT (policy ACCEPT)
```

```
target      prot opt source                      destination
```

```
root@5a5d382d0a5c:/# █
```

13.) Starting Task 2.B meant adding a firewall instruction using iptables. The command told the system to intercept certain incoming messages on the external network port. These messages were ping attempts trying to reach devices behind the router. Instead of letting them pass, the rule sends those specific signals straight to discard. Because the data does not target the router itself but flows through it, filtering happens in the FORWARD section. Blocking echo-requests this way stops outsiders from checking if inner machines are alive.

```
root@5a5d382d0a5c:/# ping 10.9.0.11
```

```
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
```

```
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.044 ms
```

```
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.032 ms
```

```
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.032 ms
```

```
64 bytes from 10.9.0.11: icmp_seq=4 ttl=64 time=0.042 ms
```

```
^C
```

```
--- 10.9.0.11 ping statistics ---
```

```
4 packets transmitted, 4 received, 0% packet loss, time 3077ms
```

```
rtt min/avg/max/mdev = 0.032/0.037/0.044/0.005 ms
```

```
root@5a5d382d0a5c:/# █
```

14.) From a remote system, the test ran just like planned. Ping reached the router - visible in the image - yet failed to touch any inside device. This matters. It confirms the setup behaves as required: external contact with the router stays open, while inward systems stay hidden from such probes.

15.)After that came a shift to one of the inner machines, aiming a test signal beyond the perimeter. Success followed - responses arrived from the outer zone, clear proof traffic could still flow out. What stood out was how these inward devices kept reaching external points using basic message checks. Important only because the setup demands such access stay live during shielding operations. The ability remained intact despite protective layers running full time.

```
root@3cb4824b7661:/# iptables -A FORWARD -i eth1 -p icmp --icmp-type echo-request -j ACCEPT
root@3cb4824b7661:/# iptables -A FORWARD -i eth0 -p icmp --icmp-type echo-reply -j ACCEPT
root@3cb4824b7661:/# iptables -P FORWARD DROP
root@3cb4824b7661:/# █
```

16.)After setting up the basics, I included a rule letting ICMP echo requests move through eth1 using iptables -A FORWARD -i eth1 -p icmp --icmp-type echo-request -j ACCEPT. Next came permission for returning echo replies arriving on eth0 via iptables -A FORWARD -i eth0 -p icmp --icmp-type echo-reply -j ACCEPT. The last part locked things down by applying iptables -P FORWARD DROP as the catch-all stance. Internal machines now reach external ones with pings thanks to those openings. Only matching responses make it back due to precise allowances written earlier. Anything not fitting these patterns gets stopped cold at the gate. External systems remain unable to initiate pings toward inner devices under this setup. They retain access to contact the router directly though. Every extra packet type finds itself shut out unless specifically cleared ahead of time. This matches exactly what the document outlined for controlled forwarding behavior.

```
root@5a5d382d0a5c:/# iptables -F
root@5a5d382d0a5c:/# iptables -P FORWARD ACCEPT
root@5a5d382d0a5c:/# iptables -P INPUT ACCEPT
root@5a5d382d0a5c:/# █
```

Task 2.C: Protecting Internal Servers

17.)Starting fresh meant wiping prior configurations before touching Task 2.C. A single new directive entered the system: iptables -A FORWARD -i eth0 -d 192.168.60.5 -p tcp --dport 23 -j ACCEPT. That line opens access solely to a lone internal device - specifically 192.168.60.5 - for telnet via port 23. External machines now connect exactly where intended, no more, no less. The setup matches lab expectations precisely - one allowed target, rest blocked by design.

```

root@5a5d382d0a5c:/# iptables -A FORWARD -i eth0 -d 192.168.60.5 -p tcp --dport 23
-j ACCEPT
root@5a5d382d0a5c:/# iptables -A FORWARD -i eth1 -s 192.168.60.5 -p tcp --sport 23
ACCEPT
Bad argument `ACCEPT'
Try `iptables -h' or 'iptables --help' for more information.
root@5a5d382d0a5c:/# iptables -A FORWARD -i eth1 -s 192.168.60.5 -p tcp --spor 23
-j ACCEPT
iptables v1.8.4 (legacy): invalid port/service `23-j' specified
Try `iptables -h' or 'iptables --help' for more information.
root@5a5d382d0a5c:/# iptables -A FORWARD -i eth1 -s 192.168.60.5 -p tcp --spor 23
-j ACCEPT
root@5a5d382d0a5c:/# █

```

18.) Then came the step where iptables -A FORWARD -i eth1 -s 192.168.60.5 -p tcp --sport 23 -j ACCEPT got put into place. Traffic heading out from that inside telnet machine now slips through thanks to this line. Without it, conversations fall apart since telnet insists on both sides sending data. Block the return path even once and everything halts - no handshake completes. One direction open means nothing if the other stays shut.

```

root@5a5d382d0a5c:/# iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination

    0      0 ACCEPT    tcp  --  eth0   *       0.0.0.0/0         192.168.60.5
    tcp dpt:23
    0      0 ACCEPT    tcp  --  eth1   *       192.168.60.5      0.0.0.0/0
    tcp spt:23

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination

root@5a5d382d0a5c:/# █

```

19.) A connection between two internal machines was checked using telnet. As shown in the image, data passed through without issues. This means systems inside the network could still talk freely among themselves. Traffic moving within stayed unblocked, proving the firewall did not mistakenly cut off local links.

```
root@6223256825f1:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
3cb4824b7661 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.15.0-139-generic x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

This system has been minimized by removing packages and content that are not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software; the exact distribution terms for each program are described in the individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.

```
seed@3cb4824b7661:~$ █
```

20.) From an external machine, I checked if connections could reach the inside servers. Only 192.168.60.5 responded - this one had clearance under the firewall policy; others stayed unreachable. That shows the rules are picking precisely who gets through, just like the job demanded. Meanwhile, inner machines couldn't phone home outward, which lines up too with what needed doing.

21.)

```
root@5a5d382d0a5c:/# iptables -F
root@5a5d382d0a5c:/# iptables -P FORWARD ACCEPT
root@5a5d382d0a5c:/# iptables -P INPUT ACCEPT
root@5a5d382d0a5c:/# iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target    prot opt in     out     source                   destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target    prot opt in     out     source                   destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target    prot opt in     out     source                   destination

root@5a5d382d0a5c:/# █
```

Task 3: Connection Tracking and Stateful Firewall

Task 3.A: Experiment with the Connection Tracking

22.) Right away, I wiped the filter rules before testing connection tracking. Inside the seed-router container, the command `conntrack -L` showed nothing at first. Not a single entry sat in the table. That told me straight up - no active flows were being tracked by the router just then. The slate was clean, ready to go.

```
root@5a5d382d0a5c:/# conntrack -L
conntrack v1.4.5 (conntrack-tools): 0 flow entries have been shown.
root@5a5d382d0a5c:/# █
```

23.) From beyond the network edge, a signal tapped inward toward one local machine. This check aimed to spot if the device logged such signals like it does with standard connections. Even though these messages lack ongoing links unlike regular data streams, they still passed through. Could the tracker notice them anyway? That uncertainty guided the trial. Tracking typically follows established pathways, yet here - no handshake occurred. Still, entries might appear regardless. Observation revealed how the system reacted when rules seemed bent slightly.

```
root@5a5d382d0a5c:/# ping 192.168.60.5 &>/dev/null &
[1] 61
root@5a5d382d0a5c:/# conntrack -L
icmp      1 29 src=192.168.60.11 dst=192.168.60.5 type=8 code=0 id=5 src=192.168.60.5 dst=192.168.60.11 type=0 code=0 id=5 mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/# █
```

24.) While ping was active, typing `conntrack -L` felt awkward since the terminal stayed busy. So instead, I sent ping to run behind the scenes using `ping 192.168.60.5 &>/dev/null &`. Only then

did the prompt become free again. With that change, the network traffic continued without blocking my next steps.

```
root@5a5d382d0a5c:/# kill %1
root@5a5d382d0a5c:/# conntrack -L
icmp      1 25 src=192.168.60.11 dst=192.168.60.5 type=8 code=0 id=5 src=192.168.60.5 dst=192.168.60.11 type=0 code=0 id=5 mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
[1]+  Terminated                  ping 192.168.60.5 &> /dev/null
root@5a5d382d0a5c:/# conntrack -L
icmp      1 2 src=192.168.60.11 dst=192.168.60.5 type=8 code=0 id=5 src=192.168.60.5 dst=192.168.60.11 type=0 code=0 id=5 mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/# conntrack -L
conntrack v1.4.5 (conntrack-tools): 0 flow entries have been shown.
root@5a5d382d0a5c:/# █
```

25.) After letting the ping run quietly, checking conntrack -L showed a single flow entry had popped up. Turns out, even without connections like TCP builds, ICMP still gets brief tracking by the kernel. Because of this, firewalls watching states can link the traffic instead of treating each packet as unrelated noise.

```
root@3cb4824b7661:/# nc -lu 9090
hello
█
```

26.) Later on, once the ping job ended, I continued watching the conntrack table at different moments. The entry lingered instead of vanishing right away. It remained active for a brief window before finally expiring. That delay matters - it shows tracked connections hold state briefly, even when data flow has ceased.

```
root@2cb892bcb1c:/# nc -u 192.168.60.5 9090
hello
█
```

27.) Twenty seconds passed before the ICMP entry vanished, at least that is what I saw. What sticks out is how one flow dropped to nothing. Memory lasts that long, no more. Timing like this shows exactly when silence follows activity.

```
root@5a5d382d0a5c:/# conntrack -L
udp       17 22 src=10.9.0.5 dst=192.168.60.5 sport=54053 dport=9090 [UNREPLIED] src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=54053 mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/# conntrack -L
udp       17 17 src=10.9.0.5 dst=192.168.60.5 sport=54053 dport=9090 [UNREPLIED] src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=54053 mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/# conntrack -L
conntrack v1.4.5 (conntrack-tools): 0 flow entries have been shown.
root@5a5d382d0a5c:/# █
```

28.) A single terminal window came up on the inside machine for the UDP test. Listening began there, waiting quietly on port 9090. The command used was `nc -l 9090` - simple enough. Packets could now arrive without warning, expected but unseen.

```
root@3cb4824b7661:/# nc -l 9090
```

29.) A signal popped up on the monitoring tool when I typed hello from another machine. It hit the UDP service at 192.168.60.5:9090 through `nc -u`. Traffic moved across the router just enough to see how stateful inspection reacted. Watching it unfold showed exactly what stayed tracked, even without handshakes.

```
root@2cb892bcbc1c:/# nc 192.168.60.5 9090
```

30.) Back at the router, after the message was sent, I kept running `conntrack -L` over and over. A UDP flow showed up in the list - clear proof something had changed. Not long ago, only empty lines filled the screen. Now there it sat: an entry where none existed before. Just like ICMP, UDP leaves traces behind. Even without handshakes or setup steps, it still appears in the table. The similarity surprised me at first. Yet there it stood, logged alongside others. No special treatment. No exceptions made.

```
root@5a5d382d0a5c:/# conntrack -L
tcp        6 431959 ESTABLISHED src=10.9.0.5 dst=192.168.60.5 sport=53024 dport=9090
           src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=53024 [ASSURED] mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/#
```

31.) Only after staring at the tracker nonstop did the UDP line finally vanish. When it left, that meant its status isn't permanent - just a short-term hold the machine clears on its own when traffic stops.

```
root@2cb892bcbc1c:/# nc 192.168.60.5 9090
hello
```

32.) After that came the TCP trial. Back at the seed-router, once everything was ready, I kept an eye on how much time passed before the TCP link disappeared from the tracking list.

```

root@5ab5d382d0a5c:/# conntrack -L
tcp      6 431969 ESTABLISHED src=10.9.0.5 dst=192.168.60.5 sport=53024 dport=9090
src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=53024 [ASSURED] mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/# conntrack -L
tcp      6 431964 ESTABLISHED src=10.9.0.5 dst=192.168.60.5 sport=53024 dport=9090
src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=53024 [ASSURED] mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/# conntrack -L
tcp      6 431958 ESTABLISHED src=10.9.0.5 dst=192.168.60.5 sport=53024 dport=9090
src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=53024 [ASSURED] mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/# conntrack -L
tcp      6 431940 ESTABLISHED src=10.9.0.5 dst=192.168.60.5 sport=53024 dport=9090
src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=53024 [ASSURED] mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/# █

```

33.) Starting the TCP check, a listener fired up on the inside machine via nc -l 9090. From the outer device, contact began toward 192.168.60.5 at port 9090 using nc. Once linked, a line of text moved across without breaking. As long as talk continued, silence didn't trigger closure. While ICMP faded fast and UDP vanished after one shot, this link held tight - alive because real back-and-forth flowed. Because TCP demands handshake and follow-through, its record persisted during full engagement.

```

root@2cb892bcb1c:/# nc 192.168.60.5 9090
hello
^C
root@2cb892bcb1c:/#
root@2cb892bcb1c:/# █

```

34.) Following manual closure of the TCP connection, I kept watching conntrack -L. The TCP flow lingered far beyond ICMP or UDP entries. Around 105 seconds passed before it disappeared, visible in the screenshot. A shifting number beside the TCP state counted backward steadily. Unlike simpler protocols, TCP follows strict steps to close properly. That delay fits its complex teardown process.

```

root@5a5d382d0a5c:/# conntrack -L
tcp      6 431940 ESTABLISHED src=10.9.0.5 dst=192.168.60.5 sport=53024 dport=9090
src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=53024 [ASSURED] mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@5a5d382d0a5c:/# conntrack -L
conntrack v1.4.5 (conntrack-tools): 0 flow entries have been shown.
root@5a5d382d0a5c:/# █

```

Task 3.B: Setting up a Stateful Firewall

35.) Starting on Task 3.B meant setting up a firewall that remembers connections, using the conntrack tool. One key line: `iptables -A FORWARD -p tcp -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT`. Packets tied to active or linked TCP sessions get through - no extra setup needed. That's different from before, when each return packet required its own rule. With tracking built in, the system recognizes live conversations between devices. No guesswork involved; it just sees what belongs.

```
root@5a5d382d0a5c:/# iptables -F
root@5a5d382d0a5c:/# iptables -P INPUT ACCEPT
root@5a5d382d0a5c:/# iptables -P OUTPUT ACCEPT
root@5a5d382d0a5c:/# iptables -P FORWARD ACCEPT
root@5a5d382d0a5c:/# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
12: eth0@if13: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP g
roup default
    link/ether 02:42:0a:09:00:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.11/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
16: eth1@if17: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP g
roup default
    link/ether 02:42:c0:a8:3c:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.60.11/24 brd 192.168.60.255 scope global eth1
        valid_lft forever preferred_lft forever
root@5a5d382d0a5c:/# █
```

36.) After that, one specific internal server got permission to accept fresh external telnet sessions. A setup similar to the PDF's method for SYN packets shaped this part. Still, changes matched exactly what Task 2.C asked. Only at this point does the firewall begin tracking connections properly. From now on, ongoing conversations pass through using the ESTABLISHED,RELATED filter without extra rules.

```
root@5a5d382d0a5c:/# iptables -A FORWARD -p tcp -m conntrack --ctstate ESTABLISHED
,RELATED -j ACCEPT
root@5a5d382d0a5c:/# iptables -A FORWARD -p tcp -i eth0 -d 192.168.60.5 --dport 23
--syn -m conntrack --ctstate NEW -j ACCEPT
root@5a5d382d0a5c:/# iptables -A FORWARD -p tcp -i eth1 -s 192.168.60.0/24 --syn -
m conntrack --ctstate NEW -j ACCEPT
root@5a5d382d0a5c:/# iptables -P FORWARD DROP
root@5a5d382d0a5c:/# █
```

37.) Inside computers now reach outward servers too, after my update. The PDF demands this shift in Task 3.B, different from earlier limits in Task 2.C where inward access blocked such moves. A new line lets local devices begin talks beyond the network perimeter. With it, outgoing traffic gets clearance even though incoming traffic stays under tight control. Design adjusts but defense does not loosen from outer threats.

```

root@2cb892bcbclc:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
3cb4824b7661 login: ^CConnection closed by foreign host.
root@2cb892bcbclc:/# telnet 192.168.60.6
Trying 192.168.60.6...
^C
root@2cb892bcbclc:/# telnet 192.168.60.7
Trying 192.168.60.7...
^C
root@2cb892bcbclc:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
2cb892bcbclc login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the

```

38.)One thing happened after changing the default FORWARD rule to DROP - the setup felt lighter, somehow neater. Connection tracking took care of return traffic without needing extra lines for each direction, so fewer rules were needed overall. This simplicity stands out when comparing stateful methods to older, rigid filters. Tracking active connections means the system must remember ongoing sessions, demanding more resources than basic packet checks. Memory and processing needs go up because decisions rely on context, not just fixed patterns.

```

root@5a5d382d0a5c:/# iptables -F
root@5a5d382d0a5c:/# iptables -P FORWARD ACCEPT
root@5a5d382d0a5c:/# █

```

Task 4: Limiting Network Traffic

39.) Starting with Task 4, attention shifted toward managing traffic flow through rate limits.

Inside the router container, a new iptables rule appeared: `-A FORWARD -s 10.9.0.5 -m limit --limit 10/minute --limit-burst 5 -j ACCEPT`. That line allows forwarding from source IP 10.9.0.5 but keeps it under control - ten packets each minute on average. Right at first, up to five packets can go through without delay. After that burst passes, enforcement begins.

```
root@5a5d382d0a5c:/# iptables -A FORWARD -s 10.9.0.5 -m limit --limit 10/minute --limit-burst 5 -j ACCEPT
root@5a5d382d0a5c:/# █
```

40.) Starting with a test, I sent pings from 10.9.0.5 to 192.168.60.5 to check the rule's behavior. Right at the beginning, a few packets slipped through - thanks to burst capacity. Then, slowly, the restriction began to hold firm. It became clear the limit module doesn't slam the door instantly. Rather, over time, it meters which packets meet the criteria. Control unfolds gradually, not all at once.

```
seed@2cb892bcbclc:~$ ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.652 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.061 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.066 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=0.067 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=63 time=0.196 ms
64 bytes from 192.168.60.5: icmp_seq=6 ttl=63 time=0.070 ms
64 bytes from 192.168.60.5: icmp_seq=7 ttl=63 time=0.072 ms
64 bytes from 192.168.60.5: icmp_seq=8 ttl=63 time=0.062 ms
64 bytes from 192.168.60.5: icmp_seq=9 ttl=63 time=0.071 ms
64 bytes from 192.168.60.5: icmp_seq=10 ttl=63 time=0.064 ms
64 bytes from 192.168.60.5: icmp_seq=11 ttl=63 time=0.365 ms
64 bytes from 192.168.60.5: icmp_seq=12 ttl=63 time=0.065 ms
64 bytes from 192.168.60.5: icmp_seq=13 ttl=63 time=0.153 ms
64 bytes from 192.168.60.5: icmp_seq=14 ttl=63 time=0.123 ms
64 bytes from 192.168.60.5: icmp_seq=15 ttl=63 time=0.070 ms
64 bytes from 192.168.60.5: icmp_seq=16 ttl=63 time=0.065 ms
64 bytes from 192.168.60.5: icmp_seq=17 ttl=63 time=0.064 ms
64 bytes from 192.168.60.5: icmp_seq=18 ttl=63 time=0.071 ms
64 bytes from 192.168.60.5: icmp_seq=19 ttl=63 time=0.071 ms
64 bytes from 192.168.60.5: icmp_seq=20 ttl=63 time=0.077 ms
64 bytes from 192.168.60.5: icmp_seq=21 ttl=63 time=0.078 ms
64 bytes from 192.168.60.5: icmp_seq=22 ttl=63 time=0.061 ms
```

41.) Starting with a test, I sent pings from 10.9.0.5 to 192.168.60.5 to check the rule's behavior. Right at the beginning, a few packets slipped through - thanks to burst capacity. Then, slowly, the

restriction began to hold firm. It became clear the limit module doesn't slam the door instantly. Rather, over time, it meters which packets meet the criteria. Control unfolds gradually, not all at once.

```
seed@2cb892bcbclc:~$ ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.208 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.064 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.053 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=0.097 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=63 time=0.067 ms
64 bytes from 192.168.60.5: icmp_seq=7 ttl=63 time=0.078 ms
64 bytes from 192.168.60.5: icmp_seq=13 ttl=63 time=0.088 ms
64 bytes from 192.168.60.5: icmp_seq=19 ttl=63 time=0.096 ms
64 bytes from 192.168.60.5: icmp_seq=25 ttl=63 time=0.061 ms
^C
--- 192.168.60.5 ping statistics ---
29 packets transmitted, 9 received, 68.9655% packet loss, time 28692ms
rtt min/avg/max/mdev = 0.053/0.090/0.208/0.044 ms
seed@2cb892bcbclc:~$
```

42.) One test used just the limit rule at first. Then came another step - adding this line: `iptables -A FORWARD -s 10.9.0.5 -j DROP`. That change plays a key role since it blocks leftover traffic from 10.9.0.5 missed by the rate filter. Left out, overflow packets could slip past the limit, moving down the chain. Their fate would depend entirely on what the rest of the rules decided afterward.

```
root@5a5d382d0a5c:/# iptables -A FORWARD -s 10.9.0.5 -j DROP
root@5a5d382d0a5c:/# iptables -L -n --line-numbers
Chain INPUT (policy ACCEPT)
num target      prot opt source                destination

Chain FORWARD (policy ACCEPT)
num target      prot opt source                destination          limit: avg 10/m
1  ACCEPT        all  --  10.9.0.5                0.0.0.0/0
in burst 5
2  DROP          all  --  10.9.0.5                0.0.0.0/0

Chain OUTPUT (policy ACCEPT)
num target      prot opt source                destination
```

43.)

```
root@5a5d382d0a5c:/# iptables -F
root@5a5d382d0a5c:/# iptables -P FORWARD ACCEPT
root@5a5d382d0a5c:/#
```

Task 5: Load Balancing

44.) Over on Host 1, running `nc -luk 8080` kicked off a UDP listener. That box now waits, quietly tuned to port 8080 for incoming data. During Task 5, it acts as an inside server - reachable when the router spreads traffic around. Setting this up? It builds one of the endpoints meant to get external flow via the router.

```
root@3cb4824b7661:/# nc -lu 9090
hello
^C
root@3cb4824b7661:/# nc -l 9090
hello
root@3cb4824b7661:/# nc -luk 8080
_
```

45.) On Host 2, I ran `nc -luk 8080` - same command, same purpose. That made it act like a UDP listener too, just waiting. Suddenly, the router sees two spots where packets could land. Without multiple machines inside, spreading traffic would be pointless anyway.

```
[03/31/26] seed@VM:~/.../Labsetup$ docksh host2-192.168.60.6
root@ad29d866c553:/# nc -luk 8080
hello2
hello5
█
```

46.) Back on Host 3, another instance of `nc -luk 8080` began running - this time setting up one more UDP listener. Now each inner machine waits at port 8080. The inside portion wraps here - the router can reach all three: 192.168.60.5, then 192.168.60.6 follows, after that comes 192.168.60.7.

```
seed@3cb4824b7661:~$ nc -luk 8080
hello1
hello4
msg1
_
```

47.) Traffic arriving at port 8080 gets redirected inside the network by changing where it thinks it's going. One setup uses a counter approach - each new batch of data jumps to the next machine in line. Another picks machines without pattern, leaning on chance to balance things out over time. This rerouting happens early, right when packets arrive at the router. Instead of sticking to one target, requests land across three separate devices. Behind this shift lies DNAT, applied

during the first step of packet handling. Rules live in the nat section under something called PREROUTING. What counts is that addresses are swapped before anything moves forward.

```

seed@VM: ~/Labsetup
root@5a5d382d0a5c:~# iptables -t nat -A PREROUTING -p udp --dport 8080 -m statistic --mode nth --every 2 --packet 0 -j DNAT --to-destination 192.168.60.6:8080
root@5a5d382d0a5c:~# iptables -t nat -A PREROUTING -p udp --dport 8080 -j DNAT --to-destination 192.168.60.7:8080
root@5a5d382d0a5c:~# iptables -t nat -L -n --line-numbers
Chain PREROUTING (policy ACCEPT)
num target prot opt source destination
1 DNAT udp -- 0.0.0.0/0 0.0.0.0/0 udp dpt:8080 statistic mode nth every 3 to:192.168.60.5:8080
2 DNAT udp -- 0.0.0.0/0 0.0.0.0/0 udp dpt:8080 statistic mode nth every 2 to:192.168.60.6:8080
3 DNAT udp -- 0.0.0.0/0 0.0.0.0/0 udp dpt:8080 to:192.168.60.7:8080

Chain INPUT (policy ACCEPT)
num target prot opt source destination

Chain OUTPUT (policy ACCEPT)
num target prot opt source destination
1 DOCKER_OUTPUT all -- 0.0.0.0/0 127.0.0.11

Chain POSTROUTING (policy ACCEPT)
num target prot opt source destination
1 DOCKER_POSTROUTING all -- 0.0.0.0/0 127.0.0.11

Chain DOCKER_OUTPUT (1 references)
num target prot opt source destination
1 DNAT tcp -- 0.0.0.0/0 127.0.0.11 tcp dpt:53 to:127.0.0.11:33279
2 DNAT udp -- 0.0.0.0/0 127.0.0.11 udp dpt:53 to:127.0.0.11:33191

Chain DOCKER_POSTROUTING (1 references)
num target prot opt source destination
1 SNAT tcp -- 127.0.0.11 0.0.0.0/0 tcp spt:33279 to::53
2 SNAT udp -- 127.0.0.11 0.0.0.0/0 udp spt:33191 to::53
root@5a5d382d0a5c:~#

```

48.) Messages went out from Host A to the router's external IP at port 8080 through a command line tool sending raw data. On reaching the network, these packets got spotted by observing machines - Host 1, then Host 2, followed by Host 3. Not every message landed on one machine; their destinations changed each time without a fixed pattern. Distribution shifted based on how traffic spread among inner systems via preset routing logic. What happened ties directly back to what Task 5 aimed to show through live behavior rather than theory. The way the router handles flow resembles how a dispatcher guides incoming tasks, whereas internal machines respond much like workers taking on jobs. When set to nth behavior, assignments follow a clear repeating sequence, yet under random settings, placement feels looser - though eventually every machine gets its share. Over time, balance emerges even without strict rules guiding each step.

```

seed@2cb892bcb1c:~$ echo hello1 | nc -u 10.9.0.11 8080
hello
^C
seed@2cb892bcb1c:~$ echo hello2 | nc -u 10.9.0.11 8080
^C
seed@2cb892bcb1c:~$ echo hello3 | nc -u 10.9.0.11 8080
^C
seed@2cb892bcb1c:~$ echo hello4 | nc -u 10.9.0.11 8080
^C
seed@2cb892bcb1c:~$ echo hello5 | nc -u 10.9.0.11 8080
^C
seed@2cb892bcb1c:~$ echo hello6 | nc -u 10.9.0.11 8080
^C
seed@2cb892bcb1c:~$ for i in 1 2 3 4 5 6 7 8 9 10 11 12; do echo msg$i | nc -u 10.9.0.11 8080; done
^C
seed@2cb892bcb1c:~$

```